Лабораторная работа №2. Управление потоками

Вихарев Лев Александрович, М8О-214БВ-25. Вариант 3.

Цель и задание

Приобрести навыки управления потоками и их синхронизации. Отсортировать массив целых чисел параллельной сортировкой слиянием на языке C, ограничив количество потоков ключом программы. Исследовать ускорение и эффективность при изменении входных данных и числа потоков.

Алгоритм и синхронизация

Реализована итеративная сортировка слиянием. На первом проходе сливаются пары отрезков длины 1, затем длина удваивается до тех пор, пока не будет отсортирован весь массив. Проходы попеременно читают исходный и вспомогательный массивы. На каждом проходе независимые пары отрезков распределяются между потоками циклически. Каждый поток записывает только свой участок выходного массива.

Главный поток тоже выполняет работу. Создаётся не больше P−1 дополнительных потоков, поэтому общее число потоков процесса не превышает P, заданного ключом --threads. Если задач меньше, потоков используется меньше. pthread_join завершает каждый проход до начала следующего и тем самым обеспечивает синхронизацию и видимость записей. При ошибке создания все уже созданные потоки дожидаются завершения, программа сообщает об ошибке.

Время алгоритма O(n log n), дополнительная память O(n + P). На последних проходах количество независимых слияний уменьшается до одного; это ограничивает ускорение. На каждом проходе потоки создаются заново, что добавляет накладные расходы.

Запуск и проверка

Сборка в Linux: make. Пример: printf "5\n3 1 2 1 -4\n" | ./merge --threads 4. Программа читает размер и элементы из стандартного ввода; результат печатается в stdout, время — в stderr. Для генерации данных: ./merge --size 1000000 --pattern random --seed 42 --threads 4 --quiet. Ключ --quiet отключает вывод массива.

Для демонстрации потоков запустить ./merge --threads 4 --size 10000 --hold 15 --quiet в фоне, затем ps -T -p PID, где PID — номер процесса, выведенный оболочкой. --hold задерживает первый проход на указанное число секунд и включается во время измерения, поэтому для исследования не используется. Проверка: make test. Выполнено 52 проверки, включая пустой массив, дубликаты, границы int, нечетные длины, случайные данные и неверные аргументы.

Исследование

Среда: Ubuntu 24.04 в WSL2, ядро Linux 6.18.54.1, Intel Core i5-13420H, системе доступно 12 логических процессоров. Сборка GCC с оптимизацией -O2. Для каждой комбинации выполнены три запуска; приведена медиана времени сортировки по CLOCK_MONOTONIC. Генерация, вывод и проверка результата в это время не входят. Ускорение S = T1/Tp, эффективность E = S/p. В качестве T1 использована медиана того же алгоритма при одном

потоке. Полные результаты и отдельные замеры сохранены в measurements.csv.

Размер массива: 10000

Данные	Потоки	Время, с	S	E
случайные	1	0.000560	1.000	1.000
случайные	2	0.001193	0.469	0.235
случайные	4	0.001521	0.368	0.092
случайные	8	0.002799	0.200	0.025
по возрастанию	1	0.000096	1.000	1.000
по возрастанию	2	0.000789	0.122	0.061
по возрастанию	4	0.001171	0.082	0.020
по возрастанию	8	0.002888	0.033	0.004
по убыванию	1	0.000101	1.000	1.000
по убыванию	2	0.001642	0.062	0.031
по убыванию	4	0.001311	0.077	0.019
по убыванию	8	0.002713	0.037	0.005

Размер массива: 1000000

Данные	Потоки	Время, с	S	E
случайные	1	0.086322	1.000	1.000
случайные	2	0.054433	1.586	0.793
случайные	4	0.038502	2.242	0.561
случайные	8	0.028467	3.032	0.379
по возрастанию	1	0.012012	1.000	1.000
по возрастанию	2	0.010814	1.111	0.555
по возрастанию	4	0.010824	1.110	0.277
по возрастанию	8	0.016469	0.729	0.091
по убыванию	1	0.013769	1.000	1.000
по убыванию	2	0.011173	1.232	0.616
по убыванию	4	0.012005	1.147	0.287
по убыванию	8	0.015756	0.874	0.109

Размер массива: 5000000

Данные	Потоки	Время, с	S	E
случайные	1	0.556222	1.000	1.000
случайные	2	0.297355	1.871	0.935
случайные	4	0.180887	3.075	0.769
случайные	8	0.143696	3.871	0.484

Данные	Потоки	Время, с	S	E
по возрастанию	1	0.077845	1.000	1.000
по возрастанию	2	0.068084	1.143	0.572
по возрастанию	4	0.053001	1.469	0.367
по возрастанию	8	0.064711	1.203	0.150
по убыванию	1	0.110239	1.000	1.000
по убыванию	2	0.069480	1.587	0.793
по убыванию	4	0.056202	1.961	0.490
по убыванию	8	0.056755	1.942	0.243

Анализ и вывод

На 10 000 случайных чисел восемь потоков работают медленнее одного: 0,002799 с против 0,000560 с. Работа слишком мала относительно стоимости создания потоков и ожидания. На 1 000 000 случайных чисел ускорение при восьми потоках составляет 3,032, эффективность 0,379. На 5 000 000 случайных чисел — 3,871 и 0,484 соответственно.

На пяти миллионах упорядоченных элементов четыре потока дают ускорение 1,469, восемь — 1,203. При обратном порядке значения составляют 1,961 и 1,942. Время зависит от порядка данных, в том числе из-за различий в поведении ветвлений при сравнении. При большем числе потоков накладные расходы, пропускная способность памяти и уменьшение числа параллельных задач ограничивают выигрыш. Результаты относятся к конкретному запуску и зависят от загрузки системы; три повтора не дают строгой статистической оценки.